HOME > SCIENCE > VOL. 359, NO. 6374 > SUPERHUMAN AI FOR HEADS-UP NO-LIMIT POKER: LIBRATUS BEATS TOP PROFESSIONALS

RESEARCH ARTICLE

Superhuman AI for heads-up no-limit poker: Libratus beats top professionals

NOAM BROWN AND TUOMAS SANDHOLM Authors Info & Affiliations

SCIENCE • 17 Dec 2017 • Vol 359, Issue 6374 • pp. 418-424 • DOI: 10.1126/science.aao1733

29.389 308

Libratus versus humans

Pitting artificial intelligence (AI) against top human players demonstrates just how far AI has come. Brown and Sandholm built a poker-playing AI called Libratus that decisively beat four leading human professionals in the two-player variant of poker called heads-up no-limit Texas hold'em (HUNL). Over nearly 3 weeks, Libratus played 120,000 hands of HUNL against the human professionals, using a three-pronged approach that included precomputing an overall strategy, adapting the strategy to actual gameplay, and learning from its opponent.

Science, this issue p. 418

Abstract

No-limit Texas hold'em is the most popular form of poker. Despite artificial intelligence (AI) successes in perfect-information games, the private information and massive game tree have made no-limit poker difficult to tackle. We present Libratus, an AI that, in a 120,000-hand competition, defeated four top human specialist professionals in heads-up no-limit Texas hold'em, the leading benchmark and long-standing challenge problem in imperfect-information game solving. Our game-theoretic approach features application-independent techniques: an algorithm for computing a blueprint for the overall strategy, an algorithm that fleshes out the details of the strategy for subgames that are reached during play, and a self-improver algorithm that fixes potential weaknesses that opponents have identified in the blueprint strategy.

SIGN UP FOR THE AWARD-WINNING *SCIENCE*ADVISER NEWSLETTER
The latest news, commentary, and research, free to your inbox daily
SIGN UP >

In recent years, the field of artificial intelligence (AI) has advanced considerably. The measure of this progress has, in many cases, been marked by performance against humans in benchmark games. AI programs have defeated top humans in checkers (*1*), chess (*2*), and Go (*3*). In these perfect-information games, both players know the exact state of the game at every point. By contrast, in imperfect-information games, some information about the state of the game is hidden from a player—for example, the opponent may hold hidden cards. Hidden information is ubiquitous in real-world strategic interactions—such as business strategy, negotiation, strategic pricing, finance, cybersecurity, and military applications—which makes research on general-purpose techniques for imperfect-information games particularly important.

Hidden information makes a game far more complex for a number of reasons. Rather than simply search for an optimal sequence of actions, an AI for imperfect-information games must determine how to balance actions appropriately, so that the opponent never finds out too much about the private information the AI has. For example, bluffing is a necessary feature in any competitive-poker strategy, but bluffing all the time would be a bad strategy. That is, the value of an action depends on the probability it is played.

Another key challenge is that different parts of the game cannot be considered in isolation; the optimal strategy for a given situation may depend on the strategy that would be played in situations that have not occurred (*4*). As a consequence, a competitive AI

CURRENT ISSUE

Thermodynamic origin of the phonon Hall effect in a honeycomb antiferromagnet
BY QINGKAI MENG, XIAOKANG LI, *ET AL.*

Electronic bound states in the continuum in a two-dimensional metal
BY MORGAN THINEL, XIONG HUANG, *ET AL.*

Wireless electrocatalytic closed-loop system for drug-free management of diabesity
BY DEBASIS MAITY, PREETAM GUHA RAY, *ET AL.*

TABLE OF CONTENTS >

LATEST NEWS

NEWS | 23 OCT 2026
Stash of hundreds of human teeth reveal surprising Maya burial practice

NEWS | 23 OCT 2026
The first dinos that only ate fruit munched like parrots

SCIENCEINSIDER | 23 OCT 2026
Kennedy to select MAHA-aligned researcher to head mental health institute, overruling NIH director

NEWS | 23 OCT 2026
Flickering signals in monkey brains hint at how we become conscious of what we see

NEWS | 22 OCT 2026
Europe's summer heat wave killed more than 50,000 people in urban areas

NEWS | 22 OCT 2026
First human death linked to recently discovered livestock virus in Europe

RELATED JOBS

Postdoctoral Scholar — Brain and Immune Therapeutics | RNA Medicines | AI-Driven Discovery
University of California, San Diego
San Diego, California

Tenure-Track Assistant Professor Gdcb
Iowa State University
Ames

Postdoctoral Scientist - Srinivasan Lab
Oklahoma Medical Research Foundation
Oklahoma City

MORE JOBS ▶

must always consider the strategy for the game as a whole.

Poker has a long history as a challenge problem for developing AIs that can address hidden information (*5*–*11*). No-limit Texas hold'em is the most popular form of poker in the world. The heads-up (that is, two-player) variant prevents opponent collusion and kingmaker scenarios where a bad player causes a mediocre player to shine and therefore allows a clear winner to be determined. Because of its large size and strategic complexity, heads-up no-limit Texas hold'em (HUNL) has been the primary benchmark and challenge problem for imperfect-information game solving for several years. No prior AI has defeated top human players in this game.

In this paper we introduce Libratus (*12*), an AI that takes a distinct approach to addressing imperfect-information games. In a 20-day, 120,000-hand competition featuring a $200,000 prize pool, it defeated top human professionals in HUNL. The techniques in Libratus do not use expert domain knowledge or human data and are not specific to poker; thus, they apply to a host of imperfect-information games.

Game-solving approach in Libratus

Libratus features three main modules:

1) The first module computes an abstraction of the game, which is smaller and easier to solve, and then computes game-theoretic strategies for the abstraction. The solution to this abstraction provides a detailed strategy for the early rounds of the game, but only an approximation for how to play in the more numerous later parts of the game. We refer to the solution of the abstraction as the blueprint strategy.

2) When a later part of the game is reached during play, the second module of Libratus constructs a finer-grained abstraction for that subgame and solves it in real time (*13*). Unlike subgame-solving techniques in perfect-information games, Libratus does not solve the subgame abstraction in isolation; instead, it ensures that the fine-grained solution to the subgame fits within the larger blueprint strategy of the whole game. The subgame solver has several key advantages over prior subgame-solving techniques (*14*–*16*). Whenever the opponent makes a move that is not in the abstraction, a subgame is solved with that action included. We call this nested subgame solving. This technique comes with a provable safety guarantee.

3) The third module of Libratus—the self-improver—enhances the blueprint strategy. It fills in missing branches in the blueprint abstraction and computes a game-theoretic strategy for those branches. In principle, one could conduct all such computations in advance, but the game tree is way too large for that to be feasible. To tame this complexity, Libratus uses the opponents' actual moves to suggest where in the game tree such filling is worthwhile.

In the following three subsections, we present these three modules in more detail.

Abstraction and equilibrium finding: Building a blueprint strategy

One solution to the problem of imperfect information is to simply reason about the entire game as a whole, rather than just pieces of it. In this approach, a solution is precomputed for the entire game, possibly using a linear program (*10*) or an iterative algorithm (*17*–*21*). For example, an iterative algorithm called counterfactual regret minimization plus (CFR+) was used to near-optimally solve heads-up limit Texas hold'em, a relatively simple version of poker, which has about 10^{13} unique decision points (*11*, *22*).

By contrast, HUNL (*23*) has 10^{161} decision points (*24*), so traversing the entire game tree even once is impossible. Precomputing a strategy for every decision point is infeasible for such a large game.

Fortunately, many of those decision points are very similar. For example, there is little difference between a bet of $100 and a bet of $101. Rather than consider every possible bet between $100 and $20,000, we could instead just consider increments of $100. This is referred to as action abstraction. An abstraction is a smaller, simplified game that retains as much as possible the strategic aspects of the original game. This drastically reduces the complexity of solving the game. If an opponent bets $101 during an actual match, then the AI may simply round this to a bet of $100 and respond accordingly (*25*–*27*). Most of the bet sizes included in Libratus's action abstraction were nice fractions or multiples of the pot [roughly determined by analyzing the most common bet sizes at various points in the game taken by prior top AIs in the Annual Computer Poker Competition (ACPC) (*28*)]. However, certain bet sizes early in the game tree were determined by an application-independent parameter-optimization algorithm that converged to a locally optimal set of bet sizes (*29*).

RECOMMENDED

RESEARCH ARTICLE | AUGUST 2019
Superhuman AI for multiplayer poker

RESEARCH ARTICLE | JANUARY 2015
Heads-up limit hold'em poker is solved

PERSPECTIVE | AUGUST 2019
AI surpasses humans at six-player poker

3 MAR 2017 | BY TONYA RILEY
Artificial intelligence goes deep to beat humans at poker

SPONSORED WEBINAR | TECHNOLOGY | 18 NOV 2026
We didn't evolve alone: How microbes shaped human adaptation

An additional form of abstraction is abstraction of actions taken by chance, that is, card abstraction, in the case of poker. Similar hands are grouped together and treated identically. Intuitively, there is little difference between a king-high flush and a queen-high flush. Treating those hands as identical reduces the complexity of the game and thus makes it computationally easier. Nevertheless, there are still differences even between a king-high flush and a queen-high flush. At the highest levels of play, those distinctions may be the difference between winning and losing. Libratus does not use any card abstraction on the first and second betting rounds. The last two betting rounds, which have a considerably larger number of states, are abstracted only in the blueprint strategy. The 55 million different hand possibilities on the third round were algorithmically grouped into 2.5 million abstract buckets, and the 2.4 billion different possibilities on the fourth round were algorithmically grouped into 1.25 million abstract buckets. However, the AI does not follow the blueprint strategy in these rounds and instead applies nested subgame solving, described in the next section, which does not use any card abstraction. Thus, each poker hand is considered individually during actual play. The card abstraction algorithm that we used was similar to that used in our prior AIs Baby Tartanian8 (*30*), which won the 2016 ACPC, and Tartanian7 (*31*–*33*), which won the 2014 ACPC. (There was no ACPC in 2015.)

Once the abstraction was constructed, we computed the blueprint strategy for Libratus by having the AI play simulated games of poker against itself (while still exploring the hypothetical outcomes of actions not chosen) using an improved version of an algorithm called Monte Carlo counterfactual regret minimization (MCCFR) (*17*, *34*, *35*), which has a long history of use in successful poker AIs (*30*, *31*, *36*, *37*). MCCFR maintains a regret value for each action. Intuitively, regret represents how much the AI regrets having not chosen that action in the past. When a decision point is encountered during self-play, the AI chooses actions with higher regret with higher probability (*38*). As more and more games are simulated, MCCFR guarantees that, with high probability, a player's average regret for any action (total regret divided by the number of iterations played) approaches zero. Thus, the AI's average strategy over all simulated games gradually improves. We will now describe the equilibrium-finding algorithm (*4*).

For each simulated game, MCCFR chooses one player (whom we refer to as the traverser) that will explore every possible action and update his regrets, while the opponent simply plays according to the strategy determined by the current regrets. The algorithm switches the roles of the two players after each game, that is, a single hand of poker. Every time either player is faced with a decision point in a simulated game, the player will choose a probability distribution over actions on the basis of regrets for those actions (which are determined by what he had learned in earlier games when he had been in that situation). For the first game, the AI has not learned anything yet and therefore uses a uniform random distribution over actions. At traverser decision points, MCCFR explores every action in a depth-first manner. At opponent decision points, MCCFR samples an action on the basis of the probability distribution. This process repeats at every decision point until the game is over and a reward is received, which is passed up. When a reward is returned by every action at a traverser decision point, MCCFR calculates the weighted average reward for that decision point on the basis of the probability distribution over actions. The regret for each action is then updated by adding the value returned by that action and subtracting the weighted average reward for the decision point. The weighted average reward is then passed up to the preceding decision point, and so on.

Our improved version of MCCFR traverses a smaller portion of the game tree on each iteration. Intuitively, there are many clearly suboptimal actions in the game and repeatedly exploring them wastes computational resources that could be better used to improve the strategy elsewhere. Rather than explore every hypothetical alternative action to see what its reward would have been, our algorithm probabilistically skips over unpromising actions that have very negative regret as it proceeds deeper into the tree during a game (*30*, *39*). In practice, this led to a speedup of MCCFR by a factor of three and allowed us to solve larger abstractions than were otherwise possible.

This skipping also mitigates the problems that stem from imperfect recall. The state-of-the-art practical abstractions in the field, including ours, are imperfect-recall abstractions where some aspects of the cards that are on the path of play so far are intentionally forgotten in order to be able to computationally afford to have a more detailed abstraction of the present state of cards (*30*–*32*, *40*). Because all decision points in a single abstract card bucket share the same strategy, updating the strategy for one of them leads to updating the strategy for all of them. This is not an issue if all of the decision points share the same optimal strategy at the solution reached, but, in practice, there are differences between their optimal strategies, and they effectively "fight" to push the bucket's strategy toward their own optimal strategy. Skipping negative-regret actions means that decision points that will never be reached in actual play will no longer have their strategies updated, thereby allowing the decision points that will actually occur during play to move the bucket's strategy closer to their optimal strategies.

We ran our algorithm on an abstraction that is very detailed in the first two rounds of HUNL, but relatively coarse in the final two rounds. However, Libratus never plays according to the abstraction solution in the final two rounds. Rather, it uses the abstract blueprint strategy in those rounds only to estimate what reward a player should expect to receive with a particular hand in a subgame. This estimate is used to determine a more precise strategy during actual play, as described in the next section.

Nested safe subgame solving

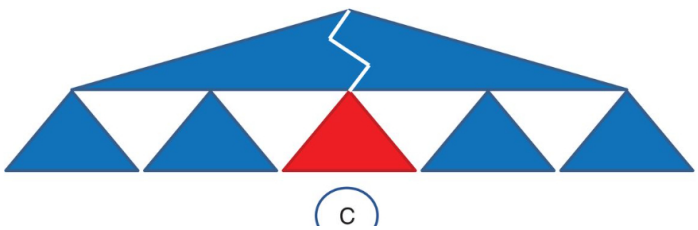
Although purely abstraction-based approaches have produced strong AIs for poker (*25*, *30*, *32*, *41*), abstraction alone has not been enough to reach superhuman performance in HUNL. In addition to abstraction, Libratus builds upon previous research into subgame solving (*14*–*16*, *42*), in which a more detailed strategy is calculated for a particular part of the game that is reached during play. Libratus features many advances in subgame solving that proved critical to achieving superhuman performance (*43*).

Libratus plays according to the abstract blueprint strategy only in the early parts of HUNL, where the number of possible states is relatively small and we can afford the abstraction to be extremely detailed. Upon reaching the third betting round, or any earlier point in the game where the remaining game tree is sufficiently small (*44*), Libratus constructs a new, more detailed abstraction for the remaining subgame and solves it in real time.

However, there is a major challenge with subgame solving in imperfect-information games: A subgame cannot be solved in isolation because its optimal strategy may depend on other, unreached subgames (*4*). Prior AIs that used real-time subgame solving addressed this problem by assuming that the opponent plays according to the blueprint strategy. However, the opponent can exploit this assumption by simply switching to a different strategy. For this reason, the technique may produce strategies that are far worse than the blueprint strategy and is referred to as unsafe subgame solving (*42*, *45*). Safe subgame-solving techniques, on the other hand, guarantee that the subgame's new strategy makes the opponent no better off no matter what strategy the opponent might use (*14*). They accomplish this by ensuring that the new strategy for the subgame fits within the overarching blueprint strategy of the original abstraction. Ensuring the opponent is no better off relative to the blueprint strategy is trivially possible because we could just reuse the blueprint strategy. However, now that the abstraction is more detailed in the subgame and we can better distinguish the strategic nuances of the subgame, it may be possible to find an improvement over the previous strategy that makes the opponent worse off no matter what cards she is holding.

We now describe Libratus's core technique for determining an improved strategy in a subgame. For exposition, we assume player 2 (P2) is determining an improved strategy against player 1 (P1). Given that P2's strategy outside the subgame is σ_2, there exists some optimal strategy σ_2^* that P2 could play in the subgame. We would like to find or approximate σ_2^* in real time. We assume that, for each poker hand P1 might have, we have a good estimate of the value P1 receives in the subgame with that hand by playing optimally against σ_2^*, even though we do not know σ_2^* itself. Although we do not know these values exactly, we can approximate them with the values P1 receives in the subgame in the blueprint strategy. We later prove that if these estimates are approximately accurate, we can closely approximate σ_2^*.

To find a strategy close to σ_2^* in the subgame using only the values from the blueprint, we create an augmented subgame (Fig. 1) that contains the subgame and additional structures. At the start of the augmented subgame, P1 is privately dealt a random poker hand. Given that P2 plays according to σ_2 before the subgame, and given P1's dealt hand, there is a particular probability distribution over what hands P2 might have in this situation. P2 is dealt a poker hand according to this probability distribution. P1 then has the choice of either entering the subgame (which is now far more detailed than in the blueprint strategy) or taking an alternative payoff that ends the augmented subgame immediately. The value of the alternative payoff is our estimate, according to the blueprint strategy, of P1's value for that poker hand in that subgame. If P1 chooses to enter the subgame, then play proceeds normally until the end of the game is reached. We can solve this augmented subgame just as we did for the blueprint strategy (*46*).



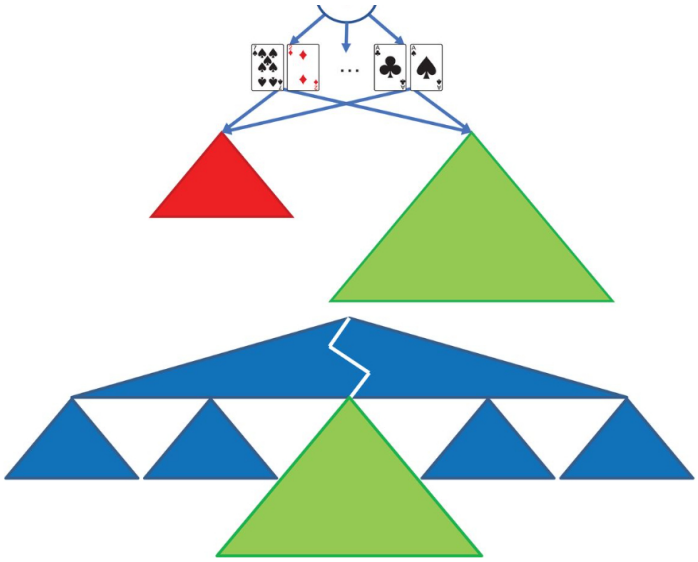


Fig. 1 Subgame solving.
(**Top**) A subgame (red) is reached during play. Blue and red indicate the blueprint strategy. The white path indicates the action sequence before the reached subgame. (**Middle**) A more-detailed strategy for that subgame is determined by solving an augmented subgame in which, on each iteration, the opponent is dealt a random poker hand and given the choice of taking the expected value of the old abstraction (red) or of playing in the new, finer-grained abstraction (green), where the strategy for both players can change. This forces Libratus to make the finer-grained strategy at least as good as that in the original abstraction against every opponent poker hand. (**Bottom**) The new strategy is substituted in place of the old one.

For any hand P1 might have, P1 can do no worse in the augmented subgame than just choosing the alternative payoff (which awards our estimate of the expected value P1 could receive against σ_2^*). At the same time, P2 can ensure that for every poker hand P1 might have, he does no better than what he could receive against σ_2^*, because P2 can simply play σ_2^* itself. Thus, any solution to the augmented subgame must do approximately as well as σ_2^*—where the approximation error depends on how far off our estimates of P1's values are. P2 then uses the solution to the augmented subgame as P2's strategy going forward.

All of this relied on the assumption that we have accurate estimates of P1's values against σ_2^*. Although we do not know these values exactly, we can approximate them with values from the blueprint strategy. We now prove that if these estimates are approximately accurate, subgame solving will produce a strategy that is close to the quality of σ_2^*. Specifically, we define the exploitability of a strategy σ_2 as how much more σ_2 would lose, in expectation, against a worst-case opponent than what P2 would lose, in expectation, in an exact solution of the full game.

Theorem 1 uses a form of safe subgame solving that we coin "Estimated-Maxmargin." We define a margin for every P1 hand in a subgame as the expected value of that hand according to the blueprint minus what P1 could earn with that hand, in expectation, by entering the more detailed subgame. Estimated-Maxmargin finds a strategy that maximizes the minimum margin among all P1 hands. It is similar to a previous technique called Maxmargin (*15*), except that technique conservatively used as the margin what P1 could earn in the subgame, in expectation, by playing a best response to P2's blueprint strategy minus what P1 could earn, in expectation, by entering the more detailed subgame.

> **Theorem 1.** Let σ_i be a strategy for a two-player zero-sum perfect-recall game, let S be a set of nonoverlapping subgames in the game, and let σ_i^* be the least-exploitable strategy that differs from σ_i only in S. Assume that for any opponent decision point (a hand in the case of poker) and any subgame in S, our estimate of the opponent's value in a best response to σ_i^* for that decision point in that subgame is off by at most Δ. Applying estimated-Maxmargin subgame solving to any subgame in S reached during play results in overall exploitability at most 2Δ higher than that of σ_i^* (*47*).

Although safe subgame-solving techniques have been known for 3 years (*14*, *15*), they were not used in practice because empirically they performed substantially worse than unsafe subgame solving (*42*) head to head (*48*). Libratus features a number of advances to subgame solving that greatly improve effectiveness:

1) Although we describe safe subgame solving as using estimates of P1 values, past techniques used upper bounds on those values (*14*, *15*). Using upper bounds guarantees that the subgame solution has exploitability no higher than the blueprint strategy. However, it tends to lead to overly conservative strategies in practice. Using estimates can, in theory, result in strategies with higher exploitability than the blueprint strategy, but Theorem 1 bounds how much higher this exploitability can be.

2) Libratus arrives at better strategies in subgames than was previously thought possible. Past techniques ensured that the new strategy for the subgame made P1 no better off in that subgame for every situation. It turns out that this is an unnecessarily strong constraint. For example, 2♠7♥ is considered the worst hand in HUNL and should be folded immediately, which ends the game. Choosing any other action would result in an even bigger loss in expectation. Nevertheless, past subgame-solving techniques would be concerned about P1 having 2♠7♥ in a subgame, which is unrealistic. Even if subgame solving resulted in a strategy that increased the value of 2♠7♥ a small amount in one subgame, that increase would not outweigh the cost of reaching the subgame (that is, the cost of not folding with 2♠7♥). Thus, P2 can allow the value of some "unimportant" P1 hands to increase in subgames, so long as the increase is small enough that it is still a mistake for P1 to reach the subgame with that hand. We accomplish this by increasing the alternative reward of P1 hands in the augmented subgame by the extra cost to P1 of reaching the subgame, that is, the size of the mistake P1 would have to make to reach that subgame with that hand. By increasing the alternative reward in the augmented subgame of these unimportant hands, P2 develops a strategy in the subgame that better defends against hands P1 might actually have (*4*).

3) Libratus crafts a distinct strategy in response to opponent bets, rather than rounding it to the nearest size in the abstraction. The optimal response to a bet of $101 is different from the optimal response to a bet of $100, but the difference is likely minor. For that reason, rounding an opponent bet of $101 to $100 is reasonable. But the optimal response to a bet of $150 is likely considerably different from the response to a bet of $100 or a bet of $200. In principle, one could simply increase the number of actions in the abstraction, perhaps by considering bets in increments of $10 rather than $100, so that the error from rounding is smaller. However, the size of the abstraction, and the time needed to solve it, increases prohibitively as more actions are added.

Therefore, rather than round to the nearest action, Libratus calculates a unique response in real time to off-tree actions, that is, an action taken by an opponent that is not in the abstraction. Libratus attempts to make the opponent no better off, no matter what hand the opponent might have, for having chosen the off-tree action rather than an action in the abstraction. It does this by generating and solving an augmented subgame following the off-tree action where the alternative payoff is the best in-abstraction action that the opponent could have taken. (The best action may differ across hands.)

Libratus repeats this for every subsequent off-tree action in a process we call nested subgame solving (see Fig. 2). Later we provide experiments that demonstrate that this technique improves the worst-case performance of poker AIs by more than an order of magnitude compared to the best technique for rounding opponent actions to a nearby in-abstraction action.

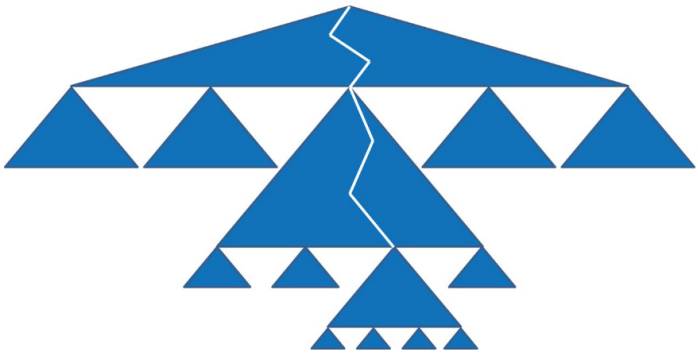


Fig. 2 A visualization of nested subgame solving.
Every time a subgame is reached during play, a more detailed abstraction is constructed and solved just for that subgame while fitting its solution within the overarching blueprint strategy.

4) Because the subgame is solved in real time, the abstraction in the subgame can also be decided in real time and change between hands. Libratus leverages this feature by changing, at the first point of subgame solving, the bet sizes it will use in that subgame and every subsequent subgame of that poker hand, thereby forcing the opponent to continually adapt to new bet sizes and strategies (*49*).

The authors of the poker AI DeepStack independently and concurrently developed an algorithm similar to nested subgame solving, which they call continual re-solving (*50*). In an internet experiment, DeepStack defeated human professionals who are not specialists in HUNL. However, DeepStack was never shown to outperform prior publicly available top AIs in head-to-head performance, whereas Libratus beats the prior leading HUNL poker AI Baby Tartanian8 by a wide margin, as we discuss later.

Like Libratus, DeepStack computes, in real time, a response to the opponent's specific bet and uses estimates rather than upper bounds on the opponent's values. It does not share Libratus's improvement of de-emphasizing hands the opponent would only be holding if she had made an earlier mistake and does not share the feature of changing the subgame action abstraction between hands.

DeepStack solves a depth-limited subgame on the first two betting rounds by estimating values at the depth limit by means of a neural network. This allows it to always calculate real-time responses to opponent off-tree actions, whereas Libratus typically plays according to its precomputed blueprint strategy in the first two rounds.

Because Libratus typically plays according to a precomputed blueprint strategy on the first two betting rounds, it rounds an off-tree opponent bet size to a nearby in-abstraction action. The blueprint action abstraction on those rounds is dense in order to mitigate this weakness. In addition, Libratus has a unique self-improvement module to augment the blueprint strategy over time, which we now introduce.

Self-improvement

The third module of Libratus is the self-improver. It enhances the blueprint strategy in the background. It fills in missing branches in the blueprint abstraction and computes a game-theoretic strategy for those branches. In principle, one could conduct all such computations in advance, but the game tree is way too large for that to be feasible. To tame this complexity, Libratus uses the opponents' actual moves to suggest where in the game tree such filling is worthwhile.

The way machine learning has typically been used in game playing is to try to build an opponent model, find mistakes in the opponent's strategy (e.g., folding too often, calling too often, and so on), and exploit those mistakes (*51*–*53*). The downside is that trying to exploit the opponent opens up oneself to being exploited. [A certain conservative family of exploitation techniques constitutes the sole exception to this downside (*51*–*53*).] For that reason, to a first approximation, Libratus did not perform opponent exploitation. Instead, it used the data of the bet sizes that the opponents used to suggest which branches should be added to the blueprint, and it then computed game-theoretic strategies for those branches in the background.

In most situations that can occur in the first two betting rounds, real-time subgame solving as used in Libratus would likely not produce a better strategy than the blueprint, because the blueprint already uses no card abstraction in those rounds and conducting subgame solving in real time so early in the game tree would require heavy abstraction in the subgame. For these reasons, Libratus plays according to the precomputed blueprint strategy in these situations. In those rounds, there are many bet sizes in the abstraction, so the error from rounding to a nearby size is small. Still, there is some error, and this could be reduced by including more bet sizes in the abstraction. In the experiment against human players described in the next section, Libratus analyzed the bet sizes in the first betting round that were most heavily used by its opponents in aggregate during each day of the competition. On the basis of the frequency of the opponent bet sizes and their distance from the closest bet size in the abstraction, Libratus chose k bet sizes for which it would try to calculate a response overnight (*54*). Each of those bet sizes for which reasonable convergence had been reached by the morning was then added to the blueprint strategy together with the newly computed strategy following that bet size. In this way, Libratus was able to progressively narrow its gaps as the competition proceeded by leveraging the humans' ability to find potential weaknesses. Furthermore, these fixes to its strategy are universal: They work against all opponents, not just the opponents that Libratus has faced.

Libratus's self-improvement comes in two forms. For one of them, when adding one of the k bet sizes, a default sibling bet size is also used during the equilibrium finding so as to not assume that the opponent necessarily only uses the bet size that will be added. For the other, a default bet size is not used. This can be viewed as more risky and even exploitative, but Libratus mitigates the risk by using that part of the strategy during play only if the opponent indeed uses that bet size most of the time (*4*).

Experimental evaluation

To evaluate the strength of the techniques used in Libratus, we first tested the overall approach of the AI on scaled-down variants of poker before proceeding to tests on full HUNL. These moderate-sized variants consisted of only two or three rounds of betting rather than four, and, at most, three bet sizes at each decision point. The smaller size of the games allowed us to precisely calculate exploitability, the distance from an optimal strategy. Performance was measured in milli–big blinds per game (mbb/game), the average number of big blinds won per 1000 games.

In the first experiment, we compared using no subgame solving, unsafe subgame solving (*42*) (in which a subgame is solved in isolation with no theoretical guarantees on

ing (42) (in which a subgame is solved in isolation with no theoretical guarantees on performance), and safe subgame solving just once upon reaching the final betting round of the game. Both players were constrained to choosing among only two different bet sizes, so off-tree actions were not an issue in this first experiment. The results are shown in Table 1. In all cases, safe subgame solving reduced exploitability by more than a factor of four relative to no subgame solving. In one case, unsafe subgame solving led to even lower exploitability, whereas in another it increased exploitability by nearly an order of magnitude more than if no subgame solving had been used. This demonstrates that although unsafe subgame solving may produce strong strategies in some games, it may also lead to far worse performance. Safe subgame solving, in contrast, reduced exploitability in all games.

Subgame-solving technique	Small two-round hold'em (mbb/game)	Large two-round hold'em (mbb/game)	Three-round hold'em (mbb/game)
No subgame solving	91.3	41.3	346
Unsafe subgame solving	5.51	397	79.3
Safe subgame solving	22.6	9.84	72.6

Table 1 Exploitability of subgame-solving techniques on smaller poker variants.
Shown is the comparison in exploitability of safe subgame-solving and unsafe subgame-solving techniques to no subgame-solving techniques for three medium-sized poker variants. Exploitability measures performance against a worst-case adversary.

In the second experiment, we constructed an abstraction of a game which only includes two of the three available bet sizes. If the opponent played the missing bet size, the AI either used action translation [in which the bet is rounded to a nearby size in the abstraction; we compared against the leading action-translation technique (27)] or nested subgame solving. The results are shown in Table 2. Nested subgame solving reduced exploitability by more than an order of magnitude relative to action translation.

Nested subgame-solving approach	Exploitability (mbb/game)
No nested subgame solving	1465
Nested unsafe subgame solving	148
Nested safe subgame solving	119

Table 2 Exploitability of nested subgame solving.
Shown is the comparison to no nested subgame solving (which instead uses the leading action translation technique) in a small poker variant.

Next, we performed experiments in full HUNL. After constructing Libratus, we tested the AI against the prior leading HUNL poker AI, our 2016 bot Baby Tartanian8, which had defeated all other poker AIs with statistical significance in the most recent ACPC (55). We report average win rates followed by the 95% confidence interval. By using only the raw blueprint strategy, Libratus lost to Baby Tartanian8 by 8 ± 15 mbb/game. Adding state-of-the-art postprocessing on the third and fourth betting rounds (31), such as eliminating low-probability actions that are likely only positive owing to insufficient time to reach convergence, led to the Libratus blueprint strategy defeating Baby Tartanian8 by 18 ± 21 mbb/game. Eliminating low-probability actions empirically leads to better performance against nonadjusting AIs. However, it also increases the exploitability of the AI because its strategy becomes more predictable. The full Libratus agent did not use postprocessing on the third and fourth betting rounds. On the first two rounds, Libratus primarily used a new, more robust form of postprocessing (4).

The next experiment evaluated nested subgame solving (with no postprocessing) using only actions that are in Baby Tartanian8's action abstraction. Libratus won by 59 ± 28 mbb/game (56). Finally, applying the nested subgame-solving structure used in the competition resulted in Libratus defeating Baby Tartanian8 by 63 ± 28 mbb/game. The results are shown in Table 3. In comparison, Baby Tartanian8 defeated the next two strongest AIs in the ACPC by 12 ± 10 and 24 ± 20 mbb/game.

Version of Libratus	Performance against Baby Tartanian8 (mbb/game)
Blueprint	−8 ± 15
Blueprint with postprocessing	18 ± 21
On-tree nested subgame solving	59 ± 28
Full nested subgame solving	63 ± 28

Full nested subgame solving	63 ± 28

Table 3 Head-to-head performance of Libratus.
Shown are results for the Libratus blueprint strategy as well as forms of nested subgame solving against Baby Tartanian8 in HUNL.

Finally, we tested Libratus against top humans. In January 2017, Libratus played against a team of four top HUNL-specialist professionals in a 120,000-hand Brains vs. Artificial Intelligence: Upping the Ante challenge match over 20 days. The participants were Jason Les, Dong Kim, Daniel McCauley, and Jimmy Chou. A prize pool of $200,000 was allocated to the four humans in aggregate. Each human was guaranteed $20,000 of that pool. The remaining $120,000 was divided among them on the basis of how much better the human did against Libratus than the worst-performing of the four humans. Libratus decisively defeated the humans by a margin of 147 mbb/game, with 99.98% statistical significance and a *P* value of 0.0002 (if the hands are treated as independent and identically distributed) [see Fig. 3; (*57*)]. It also beat each of the humans individually.

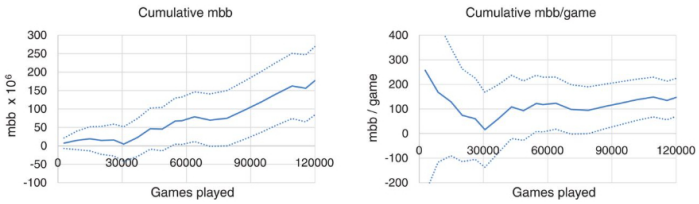


Fig. 3 Libratus performance against top humans.
Shown are the results of the 2017 Brains vs. Artificial Intelligence: Upping the Ante competition. The 95% confidence intervals (if the games are treated as independent and identically distributed) are shown as dotted lines.

Conclusions

Libratus presents an approach that effectively addresses the challenge of game-theoretic reasoning under hidden information in a large state space. The techniques that we developed are largely domain independent and can thus be applied to other strategic imperfect-information interactions, including nonrecreational applications. Owing to the ubiquity of hidden information in real-world strategic interactions, we believe the paradigm introduced in Libratus will be important for the future growth and widespread application of AI.

Acknowledgments

This material is based on research supported by the NSF under grants IIS-1718457, IIS-1617590, and CCF-1733556, and by the U.S. Army Research Office under award W911NF-17-1-0082, as well as the Extreme Science and Engineering Discovery Environment (XSEDE) computing resources and advanced user support services provided by the Pittsburgh Supercomputing Center. The Brains vs. Artificial Intelligence: Upping the Ante competition was sponsored by Carnegie Mellon University, Rivers Casino, GreatPoint Ventures, Avenue4Analytics, TNG Technology Consulting, Artificial Intelligence, Intel, and Optimized Markets, Inc. We thank B. Clayman for computing statistics of the play of our AIs against humans. The data presented in this paper are shown in the main text and supplementary materials. Additional data can be obtained from the corresponding author upon request. Because HUNL poker is played commercially, the risk associated with releasing the code outweighs the benefits. To aid reproducibility, we have included the pseudocode for the major components of our program in the supplementary materials. The technology has been exclusively licensed to Strategic Machine, Inc., and the authors have ownership interest in the company.

Supplementary Material

Summary

Supplementary Text

Figs. S1 and S2

Table S1

References (*62*–*68*)

Resources

File (aao1733_brown_sm.pdf)

Download, PDF (1.19 MB)

References and Notes

1. J. Schaeffer, *One Jump Ahead: Challenging Human Supremacy in Checkers* (Springer, 1997).
   GO TO REFERENCE • GOOGLE SCHOLAR
2. M. Campbell, A. J. Hoane Jr., F.-H. Hsu, Deep Blue. *Artif. Intell.* **134**, 57–83 (2002).
   GO TO REFERENCE • CROSSREF • WEB OF SCIENCE • GOOGLE SCHOLAR
3. D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis, Mastering the game of Go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).
   GO TO REFERENCE • CROSSREF • PUBMED • WEB OF SCIENCE • GOOGLE SCHOLAR
4. See supplementary materials for more details.
   SEE ALL REFERENCES

SHOW ALL REFERENCES

eLetters (0)

eLetters is a forum for ongoing peer review. eLetters are not edited, proofread, or indexed, but they are screened. eLetters should provide substantive and scholarly commentary on the article. Neither embedded figures nor equations with special characters can be submitted, and we discourage the use of figures and equations within eLetters in general. If a figure or equation is essential, please include within the text of the eLetter a link to the figure, equation, or full text with special characters at a public repository with versioning, such as Zenodo. Please read our Terms of Service before submitting an eLetter.

LOG IN TO SUBMIT A RESPONSE

No eLetters have been published for this article yet.

Recommended articles from TrendMD

DeepStack: Expert-level artificial intelligence in heads-up no-limit poker
Matej Moravčík, Science, 2017

Mastering the game of Stratego with model-free multiagent reinforcement learning
Julien Perolat, Science, 2022

Checkers Is Solved
Jonathan Schaeffer, Science, 2007

Mastering board games
Murray Campbell, Science, 2018

Heads-up limit hold'em poker is solved
Michael Bowling, Science, 2015

Boosting human-level AI with videogames: Mad University
Francisco Gallego, Kybernetes, 2007

GAMES, game theory and artificial intelligence
John T. Hanley, Journal of Defense Analytics and Logistics, 2021

Algorithmic Game Theory
D.M. Hutton, Kybernetes, 2008

SIMPLE EVALUATION FUNCTION
G. ZIELIŃSKI, Kybernetes, 1976

Advances in Computer Games: Many Games, Many Challenges
John Kirriemuir, Global Knowledge, Memory and Communication, 2005

Powered by TREND MD

View full text | Download PDF

Library Admin Portal
Request a Quote
Librarian FAQs

Post a Job

EurekAlert!
Science in the Classroom

Prizes and Awards

Order a Single Issue
Reprints and Permissions
TOC Alerts and RSS Feeds
Contact Us

AAAS

© 2026 American Association for the Advancement of Science. All rights reserved. AAAS is a partner of HINARI, AGORA, OARE, CHORUS, CLOCKSS, CrossRef and COUNTER. *Science* ISSN 0036-8075.

Terms of Service | Privacy Policy | Cookie Preferences | Accessibility